

# CitySense — Project Proposal

CSI 2372 • Due: Nov 29, 2025

*Team 13:*

*-Chen, Macintosh 300280808*

*-McAlister-Colacio, Shane 300511581*

*-Greene, Luca 300511570*

## Problem statement

Ottawa and nearby cities face rush-hour slowdowns, winter air advisories, and event-driven noise (construction, festivals). CitySense ingests CSV/JSON sensor streams, filters by zone/time, computes aggregates, detects incidents, and exports console + JSON reports. The design demonstrates value vs reference semantics, RAIL, exception safety, ranges, and basic concurrency.

## Unique angle (local context)

- **Local zones and events:** Downtown-Core, ByWard-Market, Glebe, Kanata-South, Gatineau-Hull. Event windows: morning/evening commute, weekend sports near Lansdowne, festivals (Bluesfest-site).
- **Seasonal realism:** Winter inversion days for PM2.5, freeze–thaw construction noise windows, snowfall congestion scenarios in simulator.
- **Ownership model clarity:** Zero-copy parse path (`SensorView`) within reader scope; promotion to `SensorRecord` at thread boundary for per-zone parallel processing.

## Scope (MVP)

- Inputs: CSV/JSON with `timestamp`, `sensor_id`, `zone_id`, `readings`; CLI flags `--inputs --zones --window --out --config`.
- Pipeline: filter → unit conversions → rolling stats → aggregates (count/mean/median/p90/p99).
- Detectors (concept-based): `TrafficCongestion`, `AirAlert`, `NoiseSpike`.
- Outputs: console hotspots + JSON report.
- Simulator: accelerated replay, pause/resume, checkpoint.
- Design task: `SensorRecord` (owning) vs `SensorView` (non-owning) with lifetime diagrams + micro-bench.

## Repo skeleton

CitySense/

```

├─ CMakeLists.txt
├─ src/
│   ├── app/{Cli.cpp, Config.hpp}
│   ├── io/{ReaderCSV.hpp, ReaderJSON.hpp, LineBuffer.hpp}
│   ├── model/{SensorRecord.hpp, SensorView.hpp}
│   ├── core/{Window.hpp, Aggregator.hpp, Detector.hpp} # Detector is a concept
│   ├── detectors/{TrafficCongestion.hpp, AirAlert.hpp, NoiseSpike.hpp}
│   ├── export/{ConsolePrinter.hpp, JsonExporter.hpp}
│   └─ sim/{Clock.hpp, Replayer.hpp, Checkpoint.hpp}
├─ data/
│   ├── sample_traffic.csv/JSON
│   ├── sample_air.CSV/JSON
│   └─ sample_noise.csv/JSON
├─ tests/{parser_tests.cpp, detector_tests.cpp, pipeline_tests.cpp}
├─ .github/workflows/ci.yml
└─ README.md

```

## Sample data (English, Ottawa-centric)

data/sample\_traffic.csv

```

timestamp,sensor_id,zone_id,speed_kmh,flow_per_min
2025-11-01T08:00:00-04:00,OT-T001,Downtown-Core,18.3,22
2025-11-01T08:01:00-04:00,OT-T001,Downtown-Core,19.1,24
2025-11-01T08:02:00-04:00,OT-T010,Kanata-South,41.0,15

```

data/sample\_air.json

```

[ { "timestamp": "2025-11-01T08:00:00-04:00", "sensor_id": "OT-A010", "zone_id": "Glebe", "pm25": 37.5, "pm10": 62.1 },
  { "timestamp": "2025-11-01T08:05:00-04:00", "sensor_id": "OT-A010", "zone_id": "Glebe", "pm25": 46.2, "pm10": 73.0 } ]

```

data/sample\_noise.csv

```

timestamp,sensor_id,zone_id,db
2025-11-01T07:30:00-04:00,OT-N007,ByWard-Market,64.0
2025-11-01T07:30:30-04:00,OT-N007,ByWard-Market,83.5

```

## Roles

*Macintosh: Role A; Shane: Role B; Greene: Role C*

- **Member A — Ingestion & Views.** Data ingress, zero-copy paths, CLI/config.
- **Member B — Analytics & Incidents.** Aggregation math, detectors, JSON/console reports.
- **Member C — Simulator & Pattern Analytics.** Realistic generator, daily/seasonal criteria, CI/tooling.

## Milestones & timeline

### Time Cost Estimation

W8 Proposal + skeleton: 8 h

W9 Alpha: 20 h

W10 Beta: 18 h

W11 RC: 10 h

Final polish + demo: 4 h

Total: 60 person-hours total ( $\approx 20$  h each)

#### W8 — Proposal + skeleton

Deliverable: 1-page plan + repo skeleton. Problem statement; sample data; risk list.

- A: CLI + config contract; Reader interface stub; .gitignore; sample CSV/JSON. — 3 h
- B: Window/Aggregate contract; Detector concept; Incident + Report schemas. — 3 h
- C: SimulatorProfile enum; Criteria struct (pm25 threshold hours, rush thresholds, evening noise). Seeded RNG stub. — 2 h

Sync: lock 3 schemas: SensorRecord/SensorView, Window, Incident; plus DailyFlags, YearSummary. No implementation debate.

#### W9 — Alpha (parsing + basic pipeline + tests)

Deliverable: parsing + basic pipeline + tests. CMake builds; CI green; value-type SensorRecord. Minute bucketing; rolling means.

- A: Streaming CSV/JSON  $\rightarrow$  SensorView; promotion rules; filters by zone/time. Tests: malformed rows, lifetime. — 8 h
- B: Minimal aggregator (count/mean), console summary. Tests: bucket math. — 7 h
- C: Simulator MVP: weekday profile (traffic/noise/air diurnal), --seed. Tiny synthetic datasets in data/. Tests: determinism with seed. — 5 h

Sync: end-to-end on tiny data; freeze public method signatures.

#### W10 — Beta (aggregations + detectors + JSON export)

Deliverable: exception safety; profiling notes. Add daily reducer and standards checks.

- A: Throughput/alloc wins (reserve, chunked parse), knobs: --batch, --reserve. Profiling notes. — 5 h
- B: Median/p90/p99; detectors ×3 (congestion, air, noise); JSON export (pretty/compact). Exception guarantees doc. — 8 h
- C: Pattern analytics: — 5 h
  - Air: flag day if any run  $\geq 120$  min with  $\text{PM}_{2.5} > \text{threshold}$ .
  - Traffic: flag day if  $\geq 2$  rush windows have mean speed  $< \text{threshold}$ .
  - Noise: flag evening if  $\geq 30$  min with  $\text{dB} \geq \text{threshold}$  in 18:00–22:00.
  - Emit daily\_flags, yearly\_summary, seasonal\_distribution. Tests for boundary cases (119 vs 120 min, 69 vs 70 dB).

Sync: one incident trace per zone validated from sample data; JSON contains daily/seasonal sections.

## W11 — RC (parallel zones + docs)

Deliverable: parallel zones; data-race free; speedup; ref-vs-value write-up.

- A: Per-zone workers (threads/futures) + SPSC queues; deterministic shutdown; TSan run. — 5 h
- B: Docs: ref-vs-value write-up; report schema doc; polish detectors thresholds. — 3 h
- C: Simulator profiles (weekday/weekend/event/snow); yearly generator; README quickstart; perf table. — 2 h

Sync: measured speedup; TSan clean; CLI demo path fixed.

## Final — Nov 29

Deliverable: polished features, tests, README,  $\leq 6$ -min demo, tag v1.0, release assets.

- A: Release build flags, packaging, CLI help polish. — 1.5 h
- B: Final tests, schema examples, sample reports. — 1.5 h
- C: Demo script, seeded replay stable outputs; upload sample yearly JSON. — 1 h

Integration and code reviews (short PRs, two per week) —  $\sim 6$  h total (spread).

CI tweaks, -fsanitize runs, flaky test fixes —  $\sim 4$  h total (spread).

# Project Governance

## Ownership boundaries

- $A \rightarrow B$ : Reader yields SensorView batches; Promoter upgrades to SensorRecord; stable Window/Aggregate API: (zone, minute\_bucket)  $\rightarrow$  stats.
- $B \rightarrow A$ : Detector concept; Incident schema; Report schema.
- $C \rightarrow B$ : Simulator stream contract; DailyFlags and YearSummary structs; criteria thresholds (Criteria).
- $B \rightarrow C$ : Required aggregates for criteria (minute means, rush windows, evening window).

## Critical path

- Alpha parse path (A)  $\rightarrow$  Aggregator contracts (B)  $\rightarrow$  Beta detectors/export (B)  $\rightarrow$  RC parallelism (A)  $\rightarrow$  Final demo (C)

## Risks

### mitigations

- **IDE/build junk in repo.** .gitignore for .vs/, build/, \*.ipch; purge history if already pushed.
- **Malformed/jittery input.** bounded diagnostics; per-file error cap.
- **Lifetime hazards with views.** keep SensorView parser-local; promote to SensorRecord at thread boundary; tests for no-dangling.
- **Data races.** Per-zone SPSC queues; immutable snapshots; ThreadSanitizer in CI.
- **Performance regressions.** Early baseline; `reserve`; fewer copies; optional PMR(polymorphic memory resource; optional, not required.) for transient buffers.
- **Schedule creep.** Limit to 3 detectors; simulator minimal but reproducible.

### Owners

- **A:** data quality, performance, lifetimes.
- **B:** correctness, incidents, reports.
- **C:** simulator realism, criteria, CI glue.
- **All:** schedule; MVP first.

### **Risk multipliers (*apply if triggered*)**

- New toolchain setup or CI breakage: +2–3 h
- Large JSON parsing corner cases: +2 h (A or B)
- Criteria edge semantics debates: +1–2 h (B + C)

## **Coordination Work principles**

- **APIs first.** Lock DTOs early; hide internals.
- **Value by default, view when local.** Views never cross threads.
- **Small PRs, daily merges.** One reviewer = the other member.
  - Make short pull requests (focused changes), open them every day, and merge to main daily after the *other* teammate reviews and approves. Result: fast feedback, fewer merge conflicts, always-releasable main.
- **Definition of done.** CI green, tests included, docs touchpoint updated.
- Views never cross threads; snapshots or values only.
  - Do not pass non-owning references (`string_view`, `spans`, pointers into parser buffers) to another thread. Other threads get either:
    - a **snapshot/value** copy (owns its memory), or
    - an **immutable struct** whose lifetime is independent of the producer.  
Rationale: avoids data races and dangling references.
- If one track finishes early, assist next dependency in the chain ( $C \rightarrow B$ ,  $A \rightarrow B$ ).

## **Flex points (swap or rebalance without churn)**

- If ingest finishes early, A helps B with detectors.
- If detectors slip, B defers p99 and ships mean/median first.
- If parallelism slips, A ships single-thread baseline + measured plan.

This split keeps interfaces stable, allows either member to swap tasks, and pushes integration early to avoid late surprises.